

LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 1
Student Name: Nimidithalli Nandu
Roll No.: A24126552100
Date: 31-08-2026

1. TITLE

Design and Implementation of Static Webpage using Semantic HTML

2. AIM

To develop a static webpage using raw HTML semantic components, tables, lists, and forms.

3. OBJECTIVE

The objectives of this experiment are:

- To practice document structuring with HTML5 semantics without CSS.
- To use forms to capture text inputs and configurations.
- To construct ordered and unordered lists.

4. THEORY

Semantic HTML tags (<header>, <nav>, <main>, <article>, <section>, <aside>, <footer>) handle the structural heavy lifting of a web page, enabling clean loading and perfect accessibility compliance.

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document.
2. Build the header section with navigation links.
3. Organize the main content into articles using semantic tags.
4. Implement a user inquiry form with input fields, select dropdown, and buttons.
5. Create an aside section to display references using lists.
6. Check rendering in a browser without CSS.

6. PROGRAM

staticwebpage.html

```
<!DOCTYPE html>  
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td align="right">
          <nav>
            <a href="#home">Home</a> &nbsp;|&nbsp;
            <a href="#articles">Articles</a> &nbsp;|&nbsp;
            <a href="#about">About</a> &nbsp;|&nbsp;
            <a href="#contact">Contact Us</a>
          </nav>
        </td>
      </tr>
    </table>
  </header>

  <hr>

  <section id="home">
    <center>
      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>

  <hr>

  <table width="100%" border="0" cellspacing="0" cellpadding="20">
    <tr valign="top">

      <td width="70%">
        <main id="articles">

          <article>
            <h2>1. Core Structural Tags</h2>
            <p>Published: July 7, 2026</p>
            <p>When you omit cascading style sheets entirely, the semantic

```

markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

`<article>`

`<h2>2. Native Interaction Elements</h2>`

`<p>Published: July 5, 2026</p>`

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

`<h3>Interactive Demonstration Forms</h3>`

`<form id="contact" action="#" method="post">`

`<fieldset>`

`<legend><b>User Inquiry Form</b></legend>`

`<p>`

`<label for="name">Full Name: </label>`

`<input type="text" id="name" name="username"`

`required placeholder="John Doe">`

`</p>`

`<p>`

`<label for="email">Email Address: </label>`

`<input type="email" id="email" name="useremail"`

`required>`

`</p>`

`<p>`

`<label for="topic">Inquiry Topic: </label>`

`<select id="topic" name="usertopic">`

`<option value="general">General`

`Feedback</option>`

`<option value="technical">Technical`

`Architecture</option>`

`<option value="academics">Academic`

`Curriculum</option>`

`</select>`

`</p>`

`<p>`

`<label for="message">Message Details:</label><br>`

`<textarea id="message" name="usermessage" rows="5"`

`cols="40" placeholder="Type your notes here..."></textarea>`

`</p>`

`<p>`

`<input type="submit" value="Submit Form Data">`

`<input type="reset" value="Clear Entries">`

`</p>`

`</fieldset>`

```

        </form>
    </article>

    </main>
</td>

<td width="30%" bgcolor="#f4f4f4">
    <aside id="about">
        <h3>Document References</h3>
        <p>Static pages map data out directly without real-time database
queries or processor overhead.</p>

        <h4>Core Benefits:</h4>
        <ul>
            <li>Instant browser painting</li>
            <li>Low memory footprints</li>
            <li>High data compatibility</li>
        </ul>

        <h4>Required Components:</h4>
        <ol>
            <li>Document Type Definition</li>
            <li>Semantic Wrapper Blocks</li>
            <li>Data Input Control Interfaces</li>
        </ol>
    </aside>
</td>

</tr>
</table>

<hr>

<footer>
    <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
    </center>
</footer>

</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
<h2> and <p>	Creates section headings and paragraphs.
<form>, <fieldset>, <legend>	Groups interactive user input fields together semantically.
<table>, <tr>, <td>	Defines a table with rows and cells for layout formatting.
, ,	Renders numbered and bulleted reference lists.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	HTML Tags	Load document	Page structure visible	Pass
TC-02	Form	Type text	Inputs accept values	Pass

9. OUTPUT

StaticWeb

[Home](#) | [Articles](#) | [About](#) | [Contact Us](#)

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

• Instant browser painting

• Low memory footprints

• High data compatibility

Required Components:

1. Document Type Definition

2. Semantic Wrapper Blocks

3. Data Input Control Interfaces

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

10. RESULT

Thus, the static webpage demonstrating raw HTML formatting and layout handling was successfully implemented and verified.

Anil Neerukonda Institute of Technology & Sciences (ANITS)
Anil Neerukonda Institute of Technology & Sciences (ANITS)

Page 5
Page 5 of 5